

Aprendizado de Máquina – IMD1101

Aula 28 – Comitês de Classificadores II

Comitê de Classificadores

Classifier Ensemble

Voting Classifiers in ML

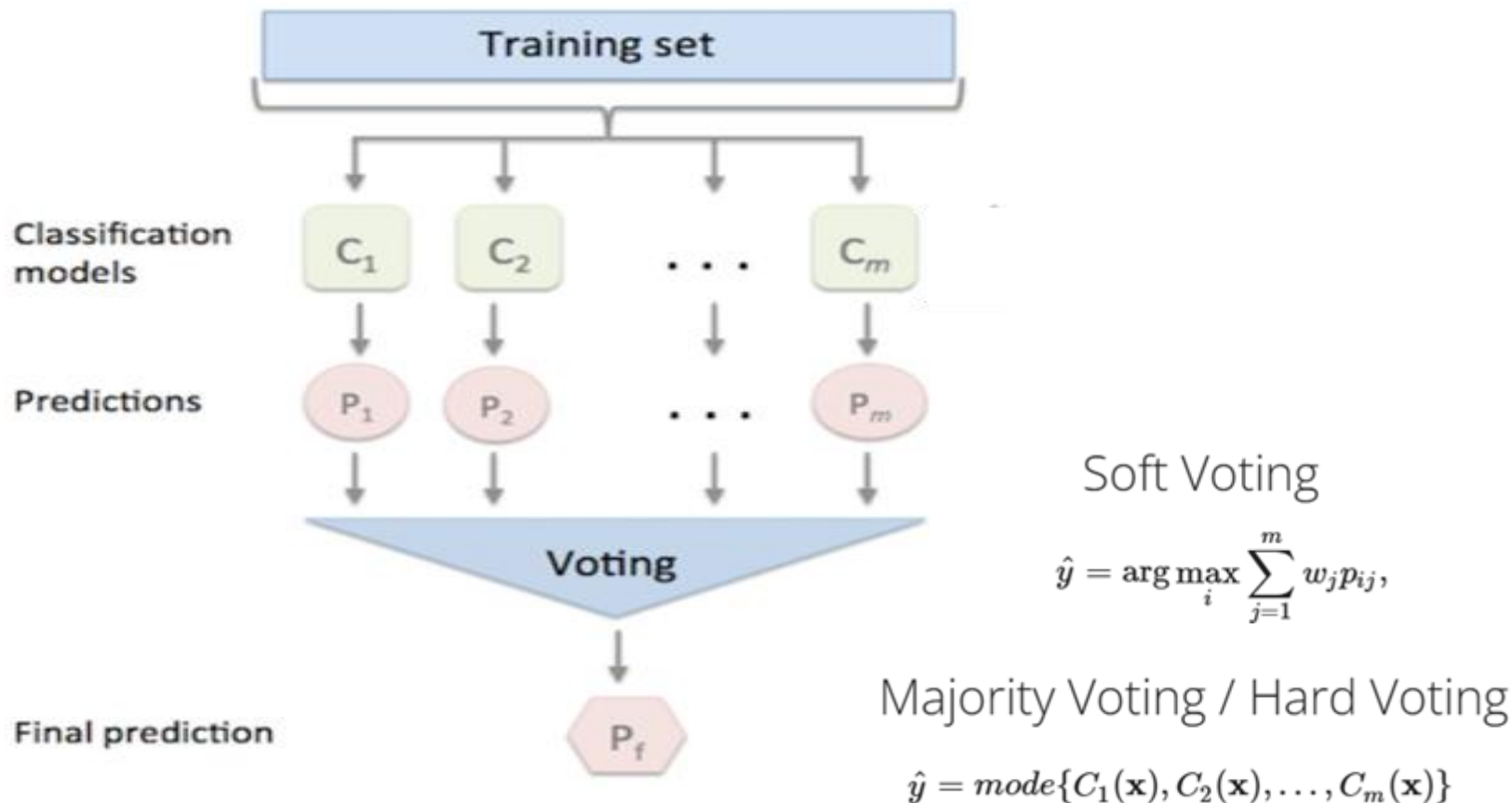
□ Voting:

- ❖ É uma técnica de comitê de classificadores que combina múltiplos classificadores e prevê a classe com base em um mecanismo de votação.

□ Existem dois tipos de Voting:

- ❖ Hard voting: A previsão final é baseada no voto majoritário dos classificadores individuais.
- ❖ Soft voting: Em vez de contar os votos, calcula a média das probabilidades previstas por cada classificador e seleciona a classe com a maior probabilidade.

Voting Classifiers in ML



Voting Classifier

- ❑ `sklearn.ensemble.VotingClassifier`
- ❑ The idea behind the Voting Classifier is to combine conceptually different machine learning classifiers and use a majority vote or the average predicted probabilities (soft vote) to predict the class labels. Such a classifier can be useful for a set of equally well performing model in order to balance out their individual weaknesses.

```
class sklearn.ensemble.VotingClassifier(estimators, *, voting='hard', weights=None, n_jobs=None, flatten_transform=True, verbose=False)
```

[\[source\]](#)

Voting Classifier

❑ Parâmetros:

- ❖ estimators: list of (str, estimator) tuples
- ❖ voting: {'hard', 'soft'}, default='hard'

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.VotingClassifier.html>

Voting Classifier

```
import pandas as pd

## Carregando o dataset
dataset = pd.read_csv('HOG_128_30x30.csv', encoding='utf-8')

# Selecionando todos os atributos com exceção da classe
X = dataset.iloc[:, :-1] # Features
y = dataset.classe        # Target variable (classe)

## Carregando o Voting e os algoritmos base (DT), (MLP), (k-NN) e (NB)
from sklearn.ensemble import VotingClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn import metrics

# Separando dataset em duas partes: treinamento e teste
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=19)
```

Voting Classifier

```
## Definição das técnicas e seus parâmetros
clf1 = MLPClassifier(hidden_layer_sizes=(100,50), activation='relu',
                    solver='adam', max_iter=1500, learning_rate_init=0.001)

clf2 = MLPClassifier(hidden_layer_sizes=(70,70), activation='relu',
                    solver='adam', max_iter=2500, learning_rate_init=0.0001)

clf3 = MLPClassifier(hidden_layer_sizes=(120), activation='identity',
                    solver='sgd', max_iter=1500, learning_rate_init=0.01)

clf4 = KNeighborsClassifier(n_neighbors=7, metric='euclidean')

clf5 = KNeighborsClassifier(n_neighbors=9, metric='euclidean')

clf6 = KNeighborsClassifier(n_neighbors=10, metric='euclidean')

clf7 = GaussianNB(priors = None, var_smoothing=1e-09)

clf8 = DecisionTreeClassifier(criterion="gini", max_depth=2)

clf9 = DecisionTreeClassifier(criterion="gini", max_depth=5)

clf10 = DecisionTreeClassifier(criterion="entropy", max_depth=9)
```


Voting Classifier

```
start = time.time()

### Configurando o Voting
vt = VotingClassifier(estimators=[('MLP01', clf1), ('MLP02', clf2), ('MLP03', clf3),
    ('kNN01', clf4), ('kNN02', clf5), ('kNN03', clf6), ('NB01', clf7), ('DT01', clf8),
    ('DT02', clf9), ('DT03', clf10)], voting='hard')

vt.fit(X_train,y_train)
y_pred = vt.predict(X_test)

end_time = time.time()

## Model Accuracy
f1_score = metrics.f1_score(y_test, y_pred, average = "weighted")
print('F1_Score: %.3f' % f1_score)
print("Tempo de execução: ", end_time - start)
```



```
F1_Score: 0.754
Tempo de execução: 36.05760669708252
```

Voting Classifier

```
start = time.time()

### Configurando o Voting
vt = VotingClassifier(estimators=[('MLP01', clf1), ('MLP02', clf2), ('MLP03', clf3),
    ('kNN01', clf4), ('kNN02', clf5), ('kNN03', clf6), ('NB01', clf7), ('DT01', clf8),
    ('DT02', clf9), ('DT03', clf10)], voting='soft')

vt.fit(X_train, y_train)
y_pred = vt.predict(X_test)

end_time = time.time()

## Model Accuracy
f1_score = metrics.f1_score(y_test, y_pred, average = "weighted")
print('F1_Score: %.3f' % f1_score)
print("Tempo de execução: ", end_time - start)
```



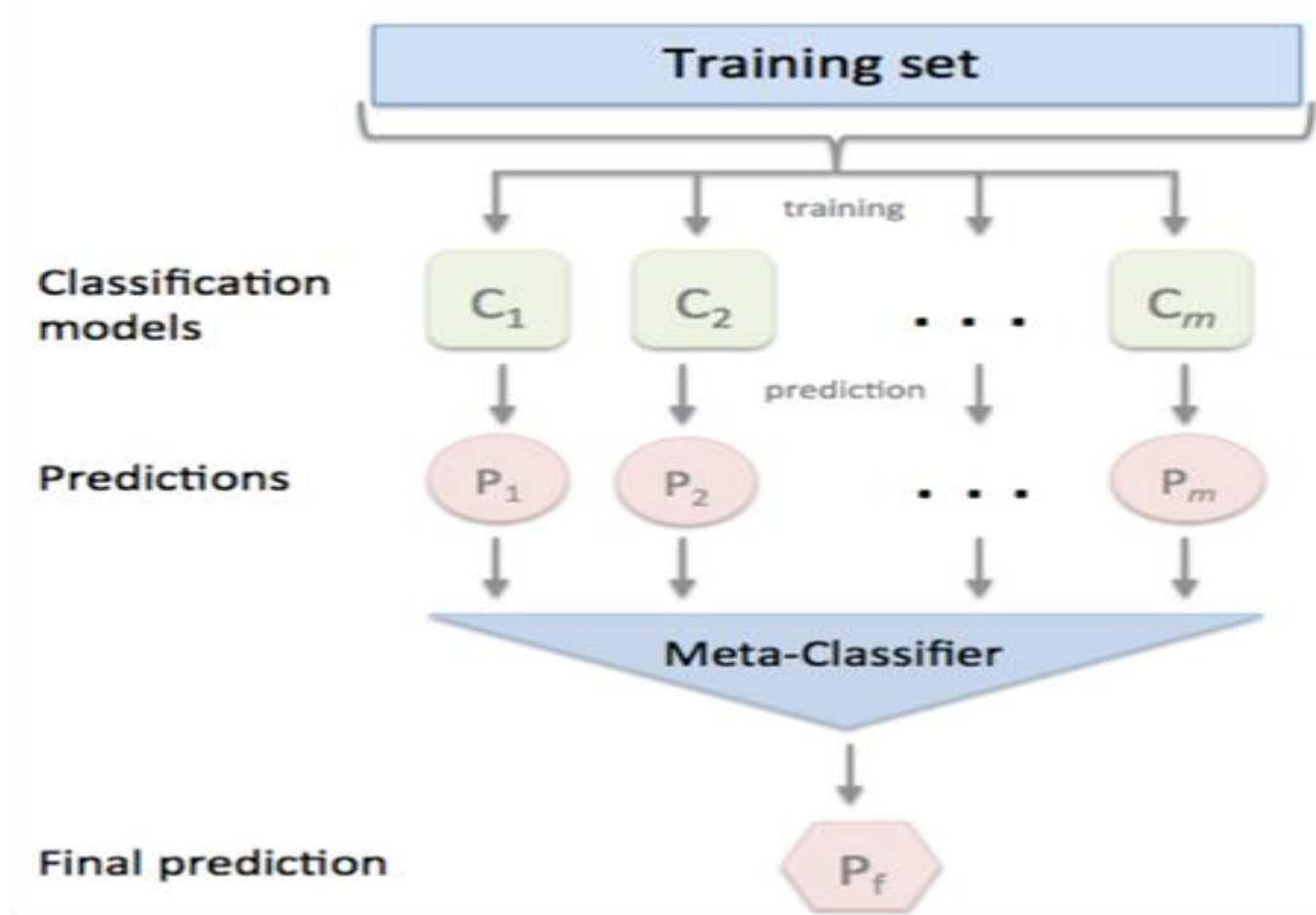
```
F1_Score: 0.788
Tempo de execução: 32.17906165122986
```

Stacking Classifiers in ML

□ Stacking:

- ❖ Técnica de ensemble mais sofisticada que envolve a combinação de diferentes tipos de modelos (classificadores base) para melhorar o desempenho.
- ❖ A ideia por trás do stacking é aproveitar os pontos fortes de vários modelos treinando um meta modelo (modelo de segundo nível) que aprende a fazer previsões com base nas saídas dos modelos base.

Stacking Classifiers in ML



Stacking Classifier

- ❑ `sklearn.ensemble. StackingClassifier`
- ❑ Stack of **estimators** with a **final classifier**. Stacked generalization consists in stacking the output of individual estimator and use a classifier to compute the final prediction. Stacking allows to use the strength of each individual estimator by using their output as input of a final estimator.

```
class sklearn.ensemble.StackingClassifier(estimators, final_estimator=None, *, cv=None, stack_method='auto',  
n_jobs=None, passthrough=False, verbose=0)
```

[\[source\]](#)

Stacking Classifier

❑ Parâmetros:

- ❖ `estimators`: list of (str, estimator)
- ❖ `final_estimator`: estimator, default=None

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html>

Stacking Classifier

```
import pandas as pd
import time

## Carregando o dataset
dataset = pd.read_csv('HOG_128_30x30.csv', encoding='utf-8')

# Selecionando todos os atributos com exceção da classe
X = dataset.iloc[:, :-1] # Features
y = dataset.classe       # Target variable (classe)

## Carregando o StackingClassifier
from sklearn.ensemble import StackingClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier

# Algoritmo para fazer a fusão
from sklearn.linear_model import LogisticRegression

from sklearn import metrics
from sklearn.metrics import confusion_matrix

# Separando dataset em duas partes: treinamento e teste
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=19)
```

Stacking Classifier

Definição das técnicas e seus parâmetros

```
estimators = [('MLP01', MLPClassifier(hidden_layer_sizes=(100,50), activation='relu',  
                                       solver='adam', max_iter=1500, learning_rate_init=0.001)),  
              ('MLP02', MLPClassifier(hidden_layer_sizes=(70,70), activation='relu',  
                                       solver='adam', max_iter=2500, learning_rate_init=0.0001)),  
              ('MLP03', MLPClassifier(hidden_layer_sizes=(120), activation='identity',  
                                       solver='sgd', max_iter=1500, learning_rate_init=0.01)),  
              ('kNN01', KNeighborsClassifier(n_neighbors=7, metric='euclidean')),  
              ('kNN02', KNeighborsClassifier(n_neighbors=9, metric='euclidean')),  
              ('kNN03', KNeighborsClassifier(n_neighbors=10, metric='euclidean')),  
              ('DT01', DecisionTreeClassifier(criterion="gini", max_depth=2)),  
              ('DT02', DecisionTreeClassifier(criterion="gini", max_depth=5)),  
              ('DT03', DecisionTreeClassifier(criterion="entropy", max_depth=9)),  
              ('NB01', GaussianNB(priors = None, var_smoothing=1e-09)) ]
```


Stacking Classifier

```
start = time.time()

stk = StackingClassifier(estimators=estimators, final_estimator=LogisticRegression())

stk.fit(X_train,y_train)
y_pred = stk.predict(X_test)

end_time = time.time()

## Model Accuracy
f1_score = metrics.f1_score(y_test, y_pred, average = "weighted")
print('F1_Score: %.3f' % f1_score)
print("Tempo de execução: ", end_time - start)
```

```
F1_Score: 0.800
Tempo de execução: 174.76846170425415
```

Dúvidas ...



Obrigado!!!

